

Backend Engineer Journey – Day 2 (Java Memory Model)

1. Java Memory has two important areas

Stack

- Fast memory
- Stores:
 - Primitive variables (`int`, `char`, `double`, etc.)
 - Local variables
 - Reference variables
 - Method call frames
- Automatically cleaned when a method finishes.

Heap

- Large memory area
- Stores all Objects
- Objects are created using the `new` keyword.
- Garbage Collector manages unused objects.

2. `new` Keyword

`new` performs three things: 1. Creates an object in Heap memory. 2. Allocates memory for that object. 3. Returns a reference to the created object.

Example:

```
Student s = new Student();
```

- Object → Heap
- Reference variable `s` → Stack
- `s` stores a reference to the object.

3. Reference Variable

A reference variable **does not store the object**.

It stores a **reference to the object**.

Example:

```
Student s = new Student();
```

Stack		Heap
s	----->	Student Object

4. Primitive vs Reference Types

Primitive Types

- int
- double
- float
- char
- boolean
- long
- short
- byte

Stored directly in Stack.

Reference Types

- Objects
- Arrays
- Strings (String objects)

Reference variable is stored in Stack. Actual object is stored in Heap.

5. Multiple References

```
Student s1 = new Student();  
Student s2 = s1;
```

- Objects = 1
- References = 2

Both references point to the same object.

Changing the object through one reference is visible through the other.

6. Reassigning a Reference

```
Student s1 = new Student();  
Student s2 = new Student();  
  
s1 = s2;
```

- Objects = 2
- References = 2
- `s1` now points to Object B.
- Object A has no references left and becomes eligible for Garbage Collection.

Golden Rule: Assignment (`=`) changes the reference, not the object.

7. Object Modification vs Reference Reassignment

Modifying Object

```
x.name = "Sameer";
```

The same object changes. All references see the updated value.

Changing Reference

```
x = new Student();
```

Only the local reference changes. The original reference remains unchanged.

8. Garbage Collection (Introduction)

An object becomes eligible for Garbage Collection when **no reachable references point to it**.

Example:

```
Student s = new Student();  
s = null;
```

The object still exists in Heap for some time, but since no reference points to it, Java can remove it later using the Garbage Collector.

9. Spring Boot Connection

When we write:

```
@Autowired  
private UserService userService;
```

Spring creates the bean (object) in Heap and injects its reference where required.

Today's Java memory concepts are the foundation of Dependency Injection.

Interview Takeaways

- Stack stores reference variables, not objects.
 - Heap stores objects.
 - `new` creates an object and returns its reference.
 - Two references can point to the same object.
 - Assignment changes references, not objects.
 - Java objects live in Heap.
 - Java uses Garbage Collection to clean unreachable objects.
-

Golden Lines (Remember Forever)

- Objects live in Heap.
- Reference variables live in Stack.
- `new` creates an object in Heap.
- Assignment (`=`) changes references, not objects.
- Modifying an object is different from changing a reference.
- If no reference points to an object, it becomes eligible for Garbage Collection.